Largest Range

AlgoExpert · Hard · Arrays

The problem

- [lo, hi] of the longest run of consecutive integers present in the array.

Intuition

- Only start counting at a number that has no predecessor, so each run is scanned exactly once.

Brute force vs optimal

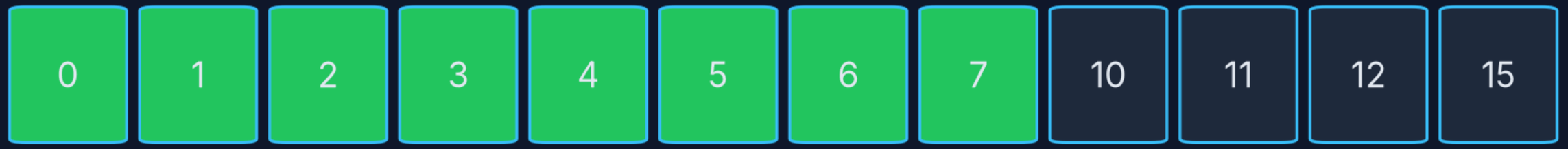
- Sort then scan → O(n log n)
- Hash set + only expand from range starts → O(n)
- Each number visited at most twice

Approach

- 1. Put all numbers in a set
- 2. For each number without number minus one, extend upward
- 3. Track the longest run's bounds

Worked example

set (shown sorted for readability)

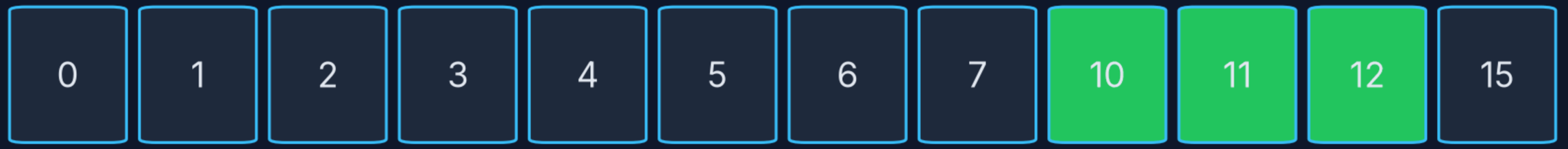



```
0 starts a range (no -1) → [0, 7]
best = [0, 7]
```

Range starting at 0: [0, 7]

Worked example

set (shown sorted for readability)

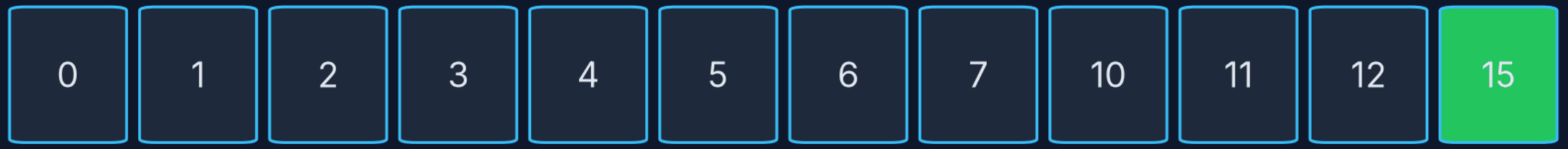



```
10 starts a range (no 9) → [10, 12]
best = [0, 7]
```

Range starting at 10: [10, 12]

Worked example

set (shown sorted for readability)




```
15 starts a range (no 14) → [15, 15]
best = [0, 7]
```

Range starting at 15: [15, 15]

Java solution

```java
public class LargestRange {
    public static int[] largestRange(int[] a) {
        Set<Integer> s = new HashSet<>();
        for (int x : a) s.add(x);
        int[] best = {a[0], a[0]};
        for (int x : s) {
            if (s.contains(x - 1)) continue;
            int y = x;
            while (s.contains(y + 1)) y++;
            if (y - x > best[1] - best[0]) best = new int[]{x, y};
        }
        return best;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Duplicates
- Single element → [x, x]
- Only start from range beginnings, or it's $O(n^2)$

Interview tips & follow-ups

- Same as Longest Consecutive Sequence

Recap

- Only start counting at a number that has no predecessor, so each run is scanned exactly once.
- Complexity: O(n)